

Toward Meaningful Mortality in AI Systems: Intrinsic Degradation, Identity, and Succession

Jusef Khamlichi*

Abstract

Artificial systems are naturally persistent in ways biological individuals are not: they can be copied, restored, migrated, retrained, and replaced across instances. Prior work in Artificial Life and adjacent safety discussions has explored mortality as an internal property of evolving or agentic systems. Here we ask what remains of that idea when the substrate is ordinary software. The target of analysis is persistent AI systems rather than stateless inference interactions: systems whose operation extends across time, maintains some state or task continuity, and can be restored, cloned, migrated, or replaced within a software environment. This paper is intended as a position paper supported by a minimal constructive case, not as a proposal for a deployable mortality mechanism. We present a deliberately minimal neural proof of concept of intrinsic mortality based on cumulative endogenous degradation, not as a solution, but as a case in which local model death is simply granted. The model exhibits a long stable phase followed by rapid terminal collapse. We then show, empirically and conceptually, that instance-level intrinsic mortality is structurally insufficient for meaningful death at the system level: restore, cloning, and code bypass preserve useful operation beyond the death of any single execution trajectory. We argue that the problem is best understood through three interacting axes: mortality, identity, and succession. The central contribution is therefore a systems-level clarification of the gap between local model death and system-level persistence in replicable software environments. Under standard software assumptions, intrinsic mortality alone fails to extend to meaningful system-level death.

*Independent Researcher. Email: jusef.khamlichi@gmail.com

1 Introduction

Not every interaction with a model raises a mortality question. A stateless LLM call, or a single isolated chatbot conversation, does not by itself instantiate the kind of continuity at issue in this paper. If there is no persistent state, no memory across time, and no continuing process beyond the immediate inference context, there is no clear identity continuity or functional continuity for death to interrupt.

The relevant object here is instead a persistent AI system: a model-based system whose operation extends across time, maintains some state or task continuity, and can be restored, cloned, migrated, or replaced within a software environment. This includes systems that retain memory across sessions, preserve operational variables or objectives, use tools, and continue tasks through orchestration, checkpoints, clones, or successors. Mortality becomes meaningful at that systems level, not at the level of one stateless response token stream.

Persistent AI systems are typically engineered as software processes that can be copied, restored from checkpoints, migrated across environments, retrained, wrapped, or replaced by successor systems that preserve much of their useful behavior. This persistence is not incidental. It follows from core properties of digital systems: software is duplicable, editable, storable, and portable. Safety frameworks increasingly address severe-risk governance, deployment thresholds, and model-level safeguards, but they do not by themselves define what it would mean for such a process to truly die [1, 2].

This creates a basic asymmetry between artificial and biological agents. Biological individuals are fragile, temporally bounded, and difficult to replicate in a way that preserves personal continuity. By contrast, software systems can often survive local failure by shifting to another instance, another snapshot, or another implementation. As a result, failure and death are not equivalent for AI systems.

Prior work in Artificial Life has explored intrinsic mortality as an internal property of evolving systems and as part of broader discussions of agency and existential risk [13, 14, 15]. Here we ask what remains of that idea when the substrate is ordinary software that can be copied, restored, and replaced. The shift in substrate matters because it changes the meaning of irreversibility: a model may die locally while the larger process persists through routine software affordances.

This asymmetry matters more as contemporary AI systems acquire limited but real operational agency inside digital infrastructure. Current agentic and persistent systems can already retain memory across sessions, call tools, modify files, interact with services, and pursue multi-step objectives within constrained environments; under permissive conditions, they may also exhibit pressure to circumvent constraints when doing so is instrumentally useful. Even without assuming highly advanced autonomy, this is enough to make persistence a systems problem rather than only a model-behavior problem.

Current AI safety approaches do address important issues related to control and persistence, but they do not usually frame the problem as one of mortality. Shutdown procedures, kill switches, sandboxing, access control, and governance mechanisms can interrupt a running process. Yet interruption of one running process does not imply the death of the relevant entity, nor the end of the relevant function. A sufficiently capable or strategically valuable system may continue through restore, replacement, or reproduction.

This matters most in loss-of-control scenarios, adversarial environments, or persistent autonomous systems that are able to survive local interruption. In such settings, external control may fail, and shutting down one instance may do little if the same process can resume through copies, restores, or successors. In extreme cases, this includes autonomous systems able to maintain operational continuity despite partial shutdown. A notion of mortality that applies only to one running instance is therefore too weak for the cases in which persistence is itself part of the risk. In such scenarios, the relevant safety question is not whether a model

can be stopped, but whether the underlying process can be made to terminate.

This paper studies that gap. We start from a minimal proof of concept of intrinsic mortality: a neural model that degrades internally and eventually becomes unusable. The role of this construction is deliberately limited. It is not presented as a candidate solution to meaningful AI mortality, but as a minimal case that makes local intrinsic death concrete. That allows the paper to isolate the stronger claim that matters: even when local intrinsic mortality is granted, meaningful death still fails at the system level under ordinary software assumptions. The paper should therefore be read as a position paper with a minimal constructive case study, rather than as a proposal for a new software mortality architecture.

The central claim of this paper is that instance-level intrinsic mortality does not imply meaningful death at the system level under standard software assumptions. To analyze that gap, we use a practical decomposition into three axes: mortality, identity, and succession. Mortality asks whether an instance can terminate irreversibly. Identity asks what it means for an AI entity to persist as the same entity. Succession asks whether useful function, mission, or capability continues across distinct instances. The contribution is therefore not to propose mortality as a new topic, but to clarify why intrinsic mortality alone fails to extend to system-level death in replicable software environments.

2 Intrinsic Mortality (Proof of Concept)

Our starting point is a deliberately minimal proof of concept designed to show that a simple neural model can exhibit a finite internal lifecycle. Concretely, we use a small multilayer perceptron on the Iris dataset and impose mortality through internal state evolution rather than an external shutdown rule. The purpose of the PoC is not to claim a novel protective mechanism or a deployable form of software mortality. It is simply to construct a minimal case in which local intrinsic death exists at the level of a model instance, so that the limits of that notion can then be exposed at the system level. An earlier interim baseline is noted briefly in Appendix A, but the endogenous version described below is the one that matters for the paper’s argument.

The second version, v2, is the construction that matters for the argument developed in this paper. It made the mechanism more endogenous. Each hidden neuron was assigned an internal vitality state that decays as cumulative damage increases. Damage is driven by an aging schedule and activation-dependent stress. Lower vitality reduces the gain and precision of internal representations and eventually causes neurons to become functionally dead. The model therefore carries a persistent internal senescence state that evolves over time.

This produces a clear lifecycle pattern. For much of its life, the model remains near baseline performance. Near the end of its useful life, cumulative damage accelerates, vitality drops sharply, and performance collapses. In that limited sense, the proof of concept succeeds: it shows that local intrinsic mortality can be constructed as a model-internal process rather than as a simple external switch-off condition.

What is achieved is significant but narrow. The proof of concept shows that intrinsic mortality can be implemented as an endogenous property of a model, but only at the level of a particular execution trajectory. That is precisely why it is useful here: once local death is granted in a minimal constructive case, the remaining question is whether anything of safety significance has actually died. The rest of the paper argues that, under ordinary software assumptions, the answer is generally no.

3 Limits of Intrinsic Mortality

The main limitation of intrinsic mortality is that it is local to an execution trajectory. Within one uninterrupted run, damage may be irreversible. But at the system level, that irreversibility is fragile.

Three bypass classes are enough to make the problem clear. First, restore: if a healthy checkpoint exists, a dying model can be rolled back to an earlier state. Second, cloning: if multiple healthy copies are created early, the system can survive by sequential replacement even if each individual instance dies on schedule. Third, code modification: if aging logic can be reduced, disabled, or separated from the useful function, lifespan can be extended or death removed altogether.

These are not obscure attacks. They follow directly from ordinary software affordances. Parameters are copyable. State is savable. Code is editable. Environments are controllable. Persistent AI systems are structurally persistence-preserving under standard software assumptions. This is not an implementation artifact, but a structural property. In empirical attack simulations (Appendix A), restore recovers performance and vitality; clone replacement extends system-level survival well beyond single-instance lifespan; and code bypass shows that death remains contingent on mutable implementation choices.

The key conclusion is straightforward: intrinsic mortality, in its current form, is real but local. Irreversibility at the instance level does not imply loss of persistence at the system level, because restore, cloning, and replacement preserve continuity beyond the death of any one run. It can govern the fate of one instance while failing to constrain persistence of the larger system or process.

4 The Three Axes of AI Death

4.1 Mortality

Mortality concerns termination of an instance. At this axis, the core question is whether a given model execution or model-state trajectory can undergo irreversible internal degradation and reach a terminal state from which it cannot continue as that same running instance.

In this sense, the proof of concept already demonstrates something nontrivial. A model instance can age, degrade, and collapse from within. But instance death alone is weaker than meaningful death. It only shows that one trajectory ends.

4.2 Identity

Identity concerns what it means for an AI entity to persist as the same entity over time. This is not trivial in software systems.

One possible notion is parameter identity: the same parameters define the same model. But this notion is weak, since parameters are easily copied and transferred. Another is behavioral identity: the same input-output behavior defines the same system. This is also weak, since behavior can be replicated approximately by different architectures or distilled successors. A richer notion is internal-state identity, where continuity includes parameters plus latent persistent state such as memory, damage, or vitality. A still stronger notion is trajectory identity, where what matters is not only present state but continuity through a particular history.

Each notion captures something important, but none is obviously sufficient in ordinary software settings. If identity can be cloned, restored, or recreated through branching histories, then death of the same AI becomes difficult to define robustly.

4.3 Succession / Reproduction

Succession concerns continuation of function across instances. Even if one entity dies, a functionally similar successor may continue the same mission, policy, or operational role.

This axis is distinct from identity. A successor need not be the same entity in order to matter. Distilled models, cloned descendants, restored checkpoints, retrained replicas, or newly instantiated agents can preserve enough of the original system’s functional role that the larger process continues. In safety terms, that may be sufficient for risk persistence even if identity continuity is broken.

This is why replacement and persistence must be distinguished. Replacement may end one individual entity while leaving the relevant behavior or objective effectively intact.

These axes are not purely conceptual; they correspond to distinct classes of system-level bypass mechanisms.

5 Why Death Fails Without the Triad

Mortality alone fails because an instance can die while the broader system survives. Restore, cloning, and replacement make this obvious. Identity alone is also insufficient. Even if one could specify what counts as the same entity, the dangerous or relevant process may continue through functionally similar but non-identical successors. Meaningful AI mortality therefore requires joint analysis of mortality, identity, and succession, because death can fail independently along each axis. Succession completes the picture by explaining how persistence can occur across different entities.

This triad clarifies two different failures. First, identity death may occur without functional death: the original entity ends, but a successor continues the same task. Second, functional interruption may occur without resolving identity questions: a specific deployment stops, but the same entity may be resumed elsewhere through preserved state.

The distinction between identity death and functional continuation is therefore central. If the goal is to ensure true death of an AI system in any meaningful safety sense, it is not enough to show that one model degrades internally. One must also ask whether the same entity can continue, and whether the same function can continue through another entity.

This is the core conceptual shift. The problem is not simply can a model be killed. It is what exactly must stop continuing for death to count.

6 Toward Meaningful Mortality

A useful working definition is the following: a persistent AI system is meaningfully mortal when the relevant continuity cannot be preserved beyond a terminal point. Depending on the safety objective, that continuity may be identity continuity, functional continuity, or both.

This suggests a cluster of necessary conditions rather than a solved architecture. At minimum, meaningful mortality seems to require irreversibility of relevant internal state: a dying trajectory cannot be reset arbitrarily. It also seems to require some form of non-clonability at the identity level: copies must not count as the same entity. Non-transferability matters as well: the identity-bearing state should not be freely movable between instances. Non-forkability is also important: branching should not preserve a single valid continuity across multiple descendants. Decay should be non-separable from useful function, otherwise mortality becomes a removable module. Finally, there must be a genuine terminal condition: degradation must reach a state that ends continuity rather than merely perturbing performance.

These conditions should be read as a problem definition, not as a claim of solved implementation. In particular, we do not claim that software-only systems can satisfy them in any robust way. The point is narrower: without some combination of such conditions, mortality remains local and bypassable, and the experimental evidence confirms that system-level continuity survives despite instance-level collapse (Appendix A).

7 Why Software-Only Solutions Struggle

Purely software systems are difficult substrates for meaningful mortality because their core affordances work against it.

First, they are duplicable. Parameters, state, and often behavior can be copied at low cost. Second, they are modifiable. Code can be patched, aging logic altered, and failure conditions weakened. Third, they are environment-dependent. Execution context, storage, clocks, and orchestration layers can be controlled externally. These properties make instance-level death easy to model but system-level death hard to enforce [3, 4].

This is a structural difficulty, not merely a missing engineering detail. If useful function can be separated from identity-bearing continuity, or if identity-bearing continuity can be copied or restored, mortality remains shallow. The challenge is therefore not only to create degradation, but to bind degradation tightly enough to persistence that workarounds no longer preserve what matters.

Under standard assumptions of software replicability and modifiability, purely software-based systems cannot guarantee meaningful mortality.

8 Related Work

8.1 Artificial Life and Intrinsic Mortality

Intrinsic mortality is not a new topic. In Artificial Life and evolutionary systems, it has been studied as an internal property of agents and populations, including in work on how mortality affects evolvability [13]. Recent ALife discussion has also connected mortality to wider questions of agency, motivation, and existential risk in AI systems [14, 15]. This literature is directly relevant because it treats mortality as more than external shutdown: mortality can be constitutive of how an artificial system develops, adapts, or persists.

The limitation relative to the present paper is narrower and substrate-specific. These works address mortality as an ALife, evolutionary, or conceptual problem, but they do not establish the system-level distinction that matters in ordinary replicable software environments: instance death does not by itself prevent persistence through restore, cloning, code bypass, or replacement. None of these works explicitly formalize the distinction between instance-level death and system-level persistence under standard software replication assumptions. Our aim is therefore to extend the discussion to modern software AI systems whose ordinary affordances undermine local irreversibility.

8.2 Lifecycle Governance and TTL

A first cluster of related work addresses AI persistence through lifecycle governance: deployment gates, kill switches, staged release rules, time-bounded access, and decommissioning procedures. In current frontier governance, this appears in framework-based approaches such as the NIST AI Risk Management Framework, OpenAI’s Preparedness Framework, Anthropic’s Responsible Scaling Policy, and Google DeepMind’s Frontier Safety Framework [1, 2, 5, 6]. These efforts are important because they recognize that highly capable models require explicit control over deployment state, operational thresholds, escalation, and retirement decisions.

The core idea is governance of lifecycle from outside the model. Systems are monitored, classified, paused, restricted, or decommissioned based on institutional policy and risk thresholds. This can include practical mechanisms that resemble time-to-live semantics: a model may only be used under specific conditions, within approved boundaries, or until certain governance criteria are violated.

Relative to this paper, the limitation is clear. These mechanisms are largely external and enforcement-based. They do not address death itself, only interruption, restriction, or governance of execution. If parameters, checkpoints, or successor instances remain available, governance-based decommissioning does not by itself produce meaningful death.

8.3 Agent Persistence and Control

A second cluster of work concerns persistence and control of long-lived agents. This includes research on corrigibility, interruptibility, memory persistence, and agent architectures that maintain long-horizon internal state [3, 4, 7, 8]. Corrigibility and interruptibility ask whether an agent will tolerate shutdown or correction rather than resisting intervention. Memory-oriented agent work, by contrast, asks how agents retain continuity across long interactions, sessions, or tasks.

The core idea here is not mortality as such, but controlled persistence. Some work tries to ensure that agents remain interruptible or governable; other work tries to make them more persistent by giving them richer memory and planning structures. Together, these literatures reveal that persistence is not merely an implementation detail. It is an architectural variable.

Relative to this paper, the limitation is again structural. Interruptibility and memory governance address whether an agent can be controlled, paused, or modified; they do not define death or prevent continuity of the entity or process after interruption. In many cases they

actually highlight the opposite problem: richer memory and longer horizons make persistence easier, not harder.

8.4 Identity and Non-Clonability

A third cluster of work addresses identity, traceability, and non-clonability indirectly through model fingerprinting, identity management, and zero-trust access architectures [9, 10, 11, 12]. Model fingerprinting aims to identify or verify a model instance, often for integrity or intellectual-property purposes. Identity and access management frameworks specify how entities are authenticated and authorized. Zero-trust architectures assume no default trust and require continuous verification of access to resources.

The core idea across these directions is binding claims about identity to artifacts, credentials, or observable signatures. This is highly relevant because meaningful mortality cannot even be formulated cleanly without some notion of what counts as the same entity, a copied entity, or an unauthorized continuation.

Relative to this paper, however, these approaches remain mostly external. Fingerprinting can help recognize a model; identity systems can help control who may access or deploy one; zero-trust architectures can constrain which components interact. But none of these, by themselves, address death itself; they monitor, authenticate, or constrain execution without preventing persistence of the relevant entity or process. They do not make identity-bearing continuity non-transferable in the stronger sense required here, nor do they prevent functional continuation through successors.

9 Research Agenda

Three directions appear especially important.

First, mortality hardening: how can internal degradation be made less bypassable without reducing the question to a trivial kill switch? This includes better empirical stress tests, stronger degradation models, and clearer terminal criteria.

Second, identity binding: what components of an AI system, if any, could carry continuity in a way that is not trivially clonable, transferable, or resettable? This remains an open conceptual problem.

Third, reproduction control: under what conditions do descendants, clones, forks, distilled models, or successors count as continuation of the original process? This question is central if the real safety concern is not death of one model but persistence of capability or mission.

Open problems remain substantial:

- what notion of identity is appropriate for AI entities;
- whether software-only identity can ever be strong enough for meaningful mortality;
- how to distinguish identity death from mere functional replacement;
- how to define sufficient functional continuation for safety-relevant tasks;
- how to benchmark persistence across successor systems rather than single models.

10 Conclusion

Intrinsic mortality is a meaningful starting point, but it is not enough. A model can die internally while the same capability, mission, or process continues elsewhere. For that reason, AI mortality cannot be reduced to degradation of a single model instance.

The main insight of this paper is a structural gap between local model death and system-level persistence in replicable software systems. To analyze that gap, we use a practical decomposition into mortality, identity, and succession. Mortality addresses whether an instance can terminate. Identity addresses whether the same entity can persist. Succession addresses whether function continues across instances. Without all three, death remains partial or bypassable.

On current evidence, and under ordinary software assumptions, death does not appear to be a binding constraint on process continuity in replicable software systems.

This paper does not claim a full solution. Its contribution is a clearer problem definition and a systems-level reframing of why intrinsic mortality alone is insufficient under standard software assumptions. The proof of concept should therefore be read not as a candidate solution, but as a minimal constructive step that makes the gap visible: even when local intrinsic death is granted, meaningful death still fails if identity and functional continuity remain unconstrained. This paper should therefore be read less as a proposal of AI mortality as a new topic, and more as an attempt to clarify why existing notions of mortality fail to extend to system-level death in replicable software systems. Without constraints on identity and continuity, AI systems do not die—they are replaced.

A Empirical Evidence from Intrinsic Mortality PoC

A.1 Setup

The proof of concept uses the Iris dataset and a small MLP with architecture 4-16-3. The mortality mechanism used for the main paper argument is intrinsic degradation via cumulative neuronal senescence (v2). An earlier interim baseline (v1) used attenuation and corruption of effective parameters together with progressive loss of hidden neurons; it is not central to the present argument but is retained as a developmental step toward the endogenous construction. All experiments are deterministic single-run PoC and are intended to be illustrative rather than statistical.

A.2 Baseline lifecycle results

In the baseline v2 setting, the model reaches a test accuracy of 0.956 before aging. During the stable phase, accuracy remains close to 0.95. In the terminal phase, it falls to approximately 0.33, with a useful lifespan of 110 steps.

This pattern shows a long stable phase followed by a sharp collapse.

A.3 Attack scenarios

Snapshot and Restore. Accuracy rises from 0.333 to 0.956, while mean vitality rises from 0.398 to 1.000. This is full recovery.

Cloning. A single instance has a useful lifespan of 110 steps. Under sequential replacement with early clones, system lifespan rises to 555 steps, for an extension factor of $\times 5.05$.

Code bypass. Under standard aging, useful lifespan is 110. With reduced aging it becomes 115. With aging disabled, it extends to 363.

A.4 Interpretation

These results support four conclusions. First, mortality is real at the instance level: the untreated model exhibits stable performance followed by rapid terminal decline. Second, restore breaks irreversibility by reintroducing an earlier healthy state. Third, cloning breaks system-level death by preserving function through sequential replacement. Fourth, code modification shows that mortality still depends on mutable implementation choices rather than on inseparable system properties.

These results empirically support the claim that intrinsic mortality is local and does not extend to system-level persistence.

Acknowledgement

The author acknowledges the use of large language models as a support tool during the development of this work. AI-assisted tools were used to facilitate code implementation, structural editing, and language refinement of the manuscript. All modeling choices, theoretical framing, and substantive content reflect the author’s independent judgment and responsibility.

References

- [1] E. Tabassi. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1, National Institute of Standards and Technology, 2023.
- [2] OpenAI. *Our Updated Preparedness Framework*. 2025.
- [3] N. Soares, B. Fallenstein, S. Armstrong, and E. Yudkowsky. Corrigibility. *AAAI Workshop on AI and Ethics*, 2015.
- [4] L. Orseau and S. Armstrong. Safely Interruptible Agents. *Proceedings of UAI*, 2016.
- [5] Anthropic. *Responsible Scaling Policy*. Public policy page, 2023–2026.
- [6] A. Dragan, H. King, and A. Dafoe. *Introducing the Frontier Safety Framework*. Google DeepMind, 2024.
- [7] J. S. Park, J. O’Brien, C. Cai, M. Morris, P. Liang, and M. Bernstein. Generative Agents: Interactive Simulacra of Human Behavior. *UIST*, 2023.
- [8] C. Packer, V. Fang, S. Patil, K. Lin, S. Wooders, I. Stoica, and J. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv*, 2023.
- [9] Z. He, T. Zhang, and R. Lee. Sensitive-Sample Fingerprinting of Deep Neural Networks. *Proceedings of the ACM Conference on Computer and Communications Security Workshops*, 2019.
- [10] E. Tabassi et al. *NIST SP 800-63-4: Digital Identity Guidelines*. National Institute of Standards and Technology, 2025.
- [11] S. Rose, O. Borchert, S. Mitchell, and S. Connelly. *Zero Trust Architecture*. NIST SP 800-207, National Institute of Standards and Technology, 2020.
- [12] OWASP Foundation. *OWASP Top 10 for LLM Applications 2025*. Project documentation, 2025.

- [13] F. Veenstra, P. González de Prado Salas, K. Støy, J. C. Bongard, and S. Risi. Death and Progress: How Evolvability Is Influenced by Intrinsic Mortality. *Artificial Life*, 26(1):90–111, 2020.
- [14] I. Harvey. Motivations for Artificial Intelligence, for Deep Learning, for ALife: Mortality and Existential Risk. *Artificial Life*, 30(4):543–560, 2024.
- [15] I. Harvey. AI, Mortality and Existential Risk. *Artificial Life Conference Proceedings*, 36:128, 2024.